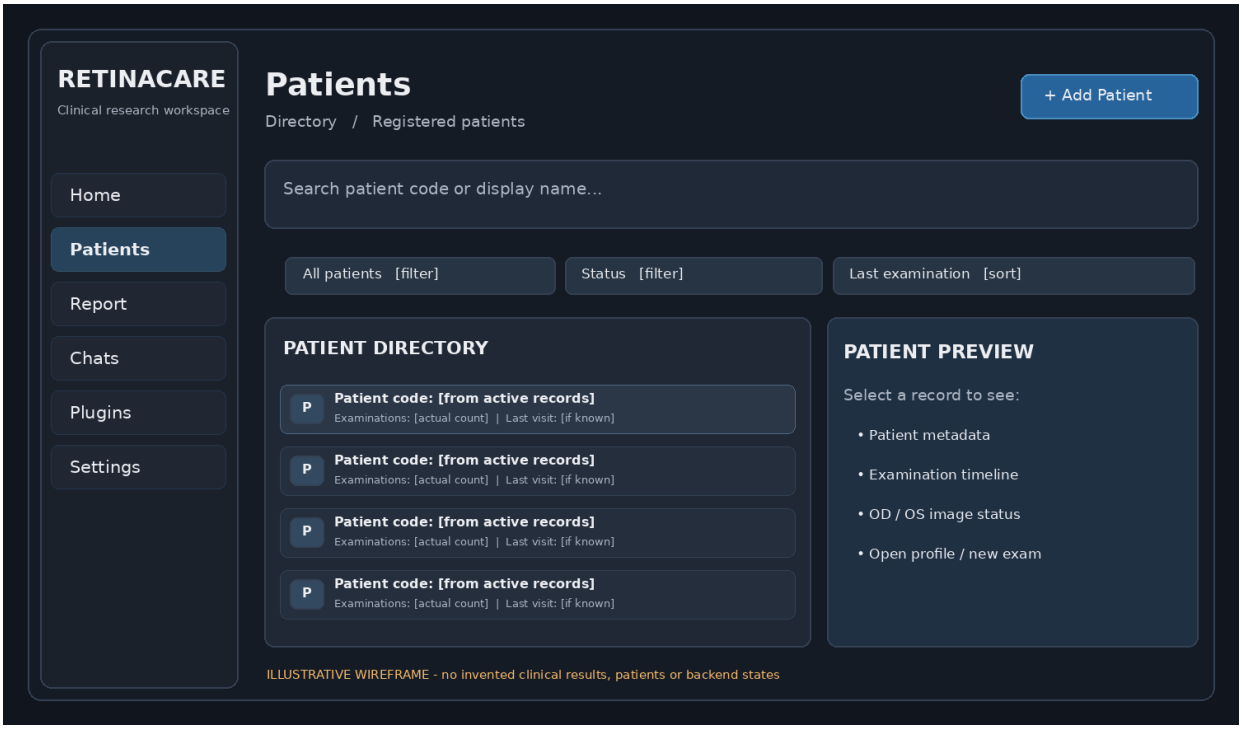
RETINACARE / ANTIGRAVITY

Task 06.1 - Separate Screens & Premium Patients Page Redesign

A screenshot-grounded native Rust GUI correction guide. Includes visual wireframes, detailed normal prompt, complete JSON prompt, testing plan, and honest acceptance criteria.

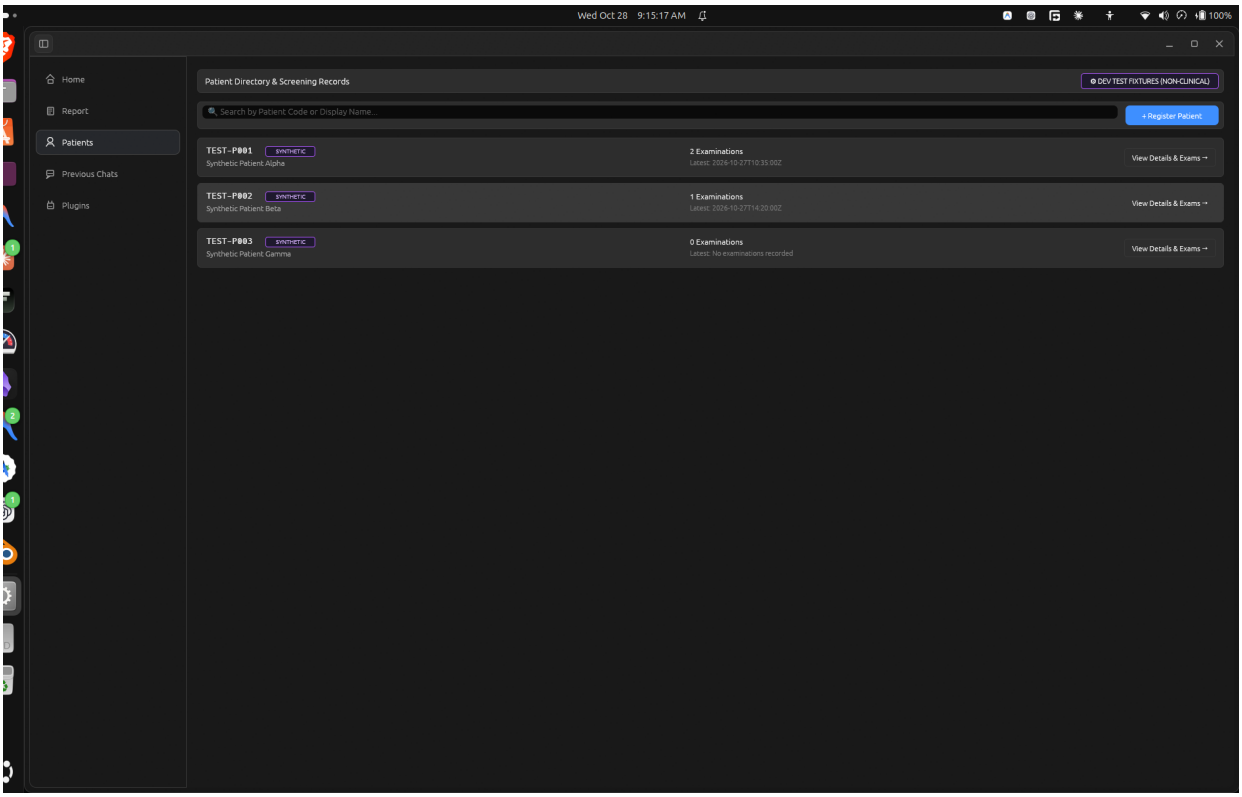


Proposed Patients Directory - an illustrative design reference, not an application screenshot.

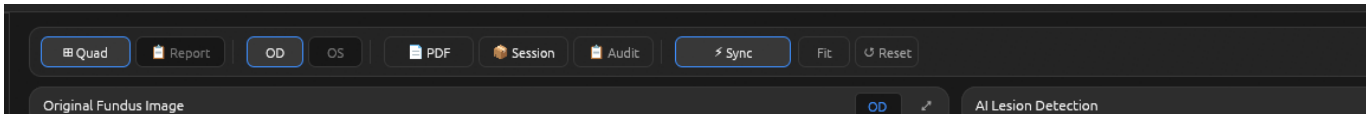
Why this correction matters: The Patients sidebar route already exists, but the shown page has three synthetic list rows and a large unused canvas. The Report toolbar mixes viewer settings with PDF, Session and Audit workflows. This task separates navigation destinations and improves information architecture without adding a web frontend.

Current evidence: supplied RetinaCare GUI screenshots and latest Task 05 report. The actual repository state must be audited by Antigravity before changing source.

1. Screenshot audit - what is actually visible



Screenshot B: Existing Patients sidebar route. The list is present; the design lacks a real master-detail workflow and leaves most of the screen unused.

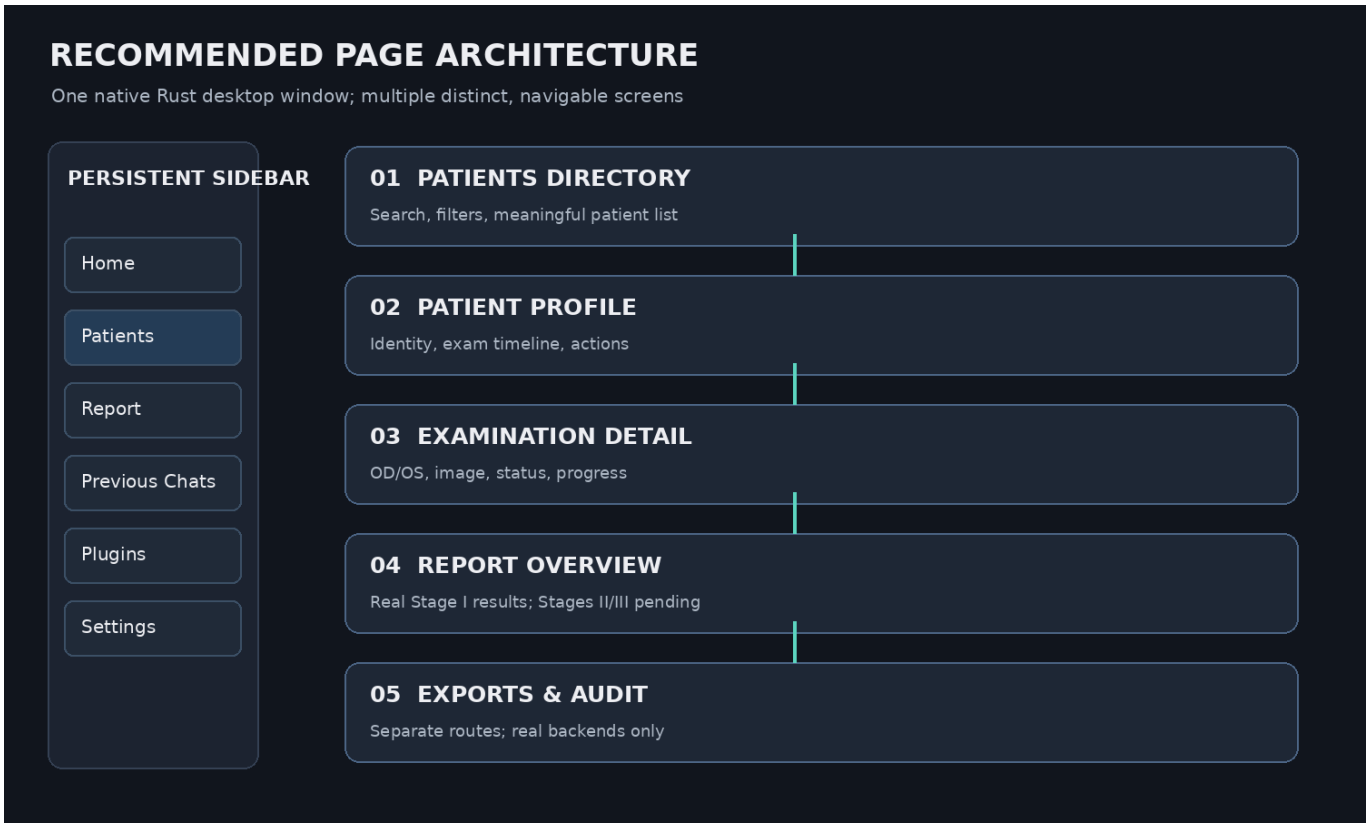


Screenshot A: Report toolbar with Quad / Report and PDF / Session / Audit competing with viewer controls.

Do not misdiagnose the problem

- Patients is already a separate sidebar destination. The missing element is a dedicated Patient Profile and Examination Detail experience, not another top-level sidebar button.
- PDF, Session and Audit are workflows. They deserve meaningful routed screens; short confirmations may still use dialogs.
- Existing synthetic patients are test fixtures. Never present these as real clinic records.

2. Proposed screen map and user journey

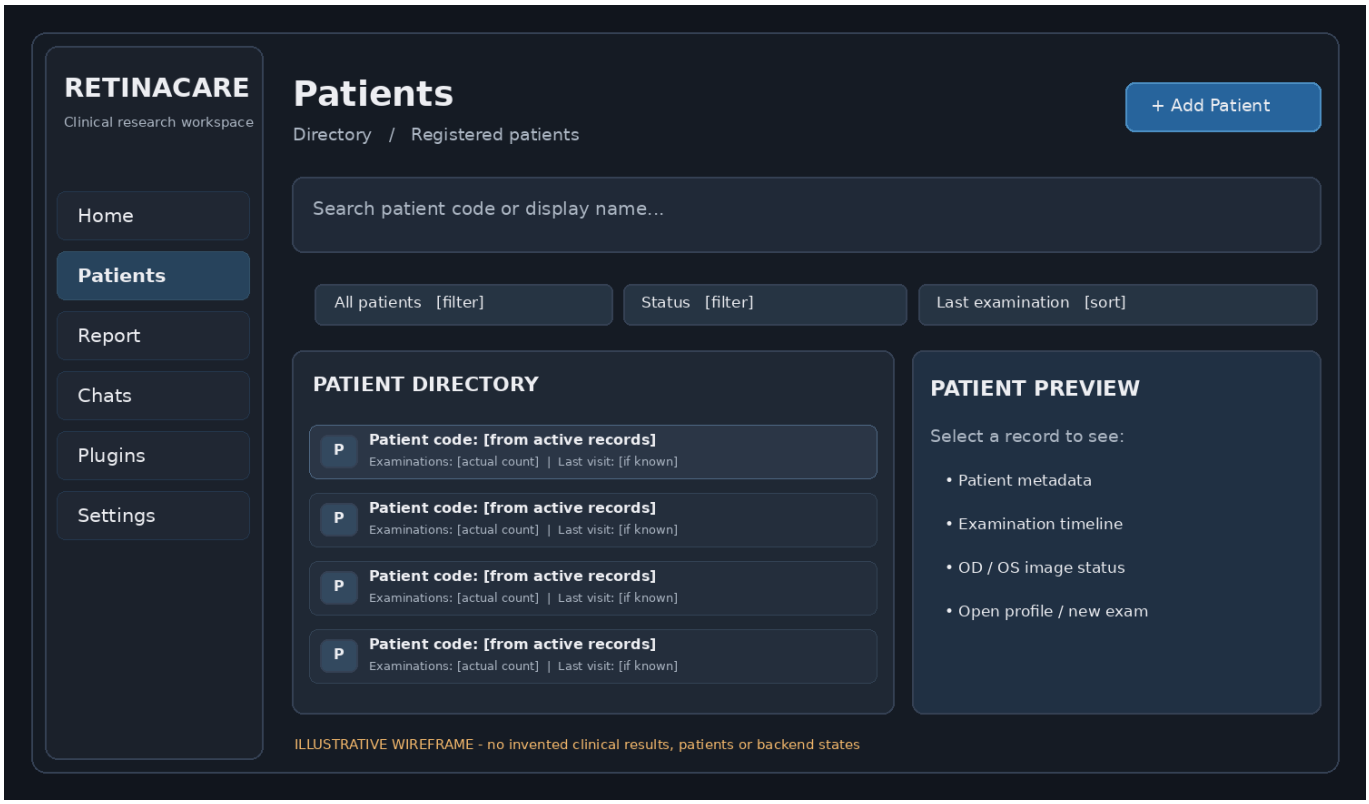


An explicit route hierarchy inside the same egui/eframe application. The left sidebar remains visible.

Example route flow: Patients Directory > Patient Profile > Examination Detail > Report Overview or Quad Viewer > optional Export or Audit destination.

Each transition must preserve patient/examination identity and provide Back / breadcrumbs. A distinct application screen is NOT a separate Linux desktop window.

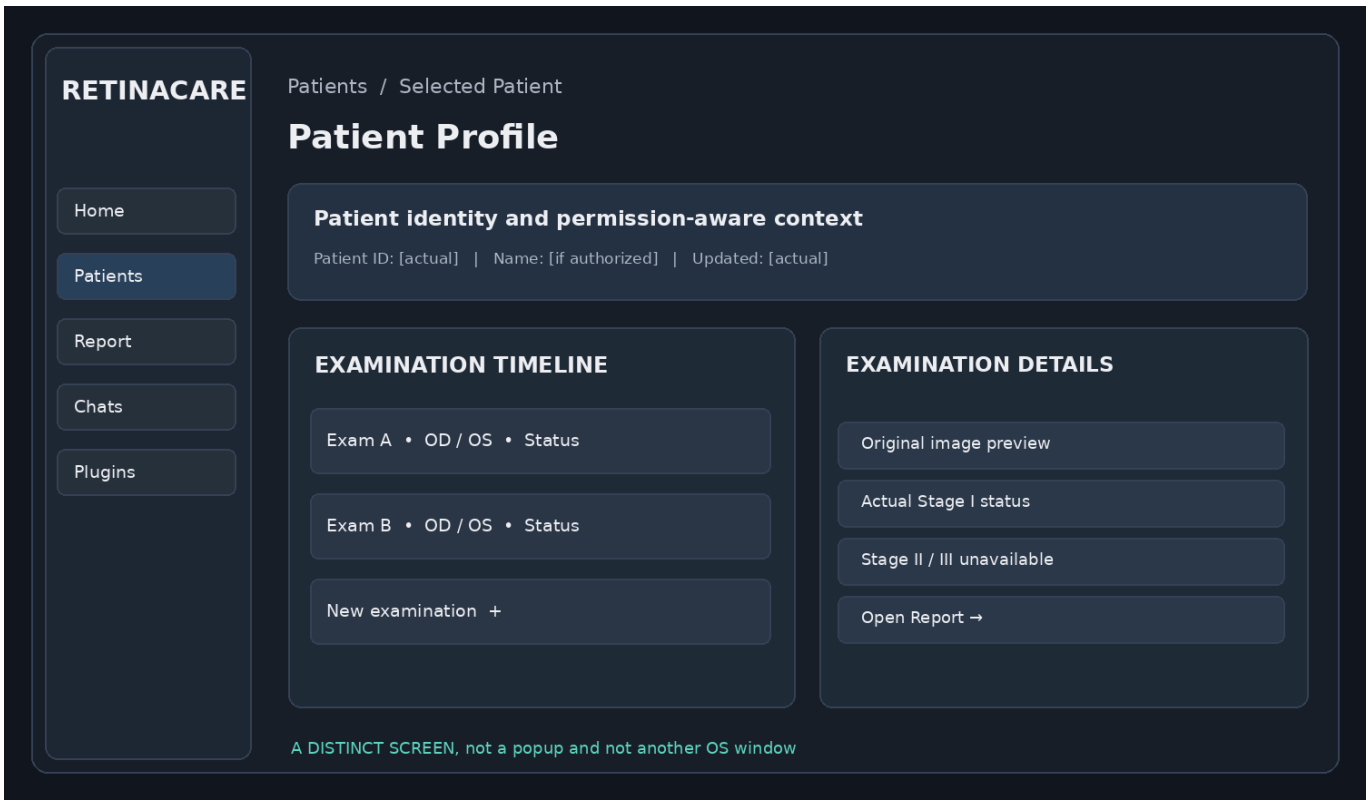
3. Design reference - Patients Directory



Master-detail Patients layout with a real record list, relevant filters and contextual preview. Placeholder labels must be replaced with actual data only.

- Strong header, concise actions, readable row density and restrained accent colors.
- Use real patient directory fields and real counts; no invented metrics.
- At narrow width, place preview below the list or navigate to the profile; no clipping.

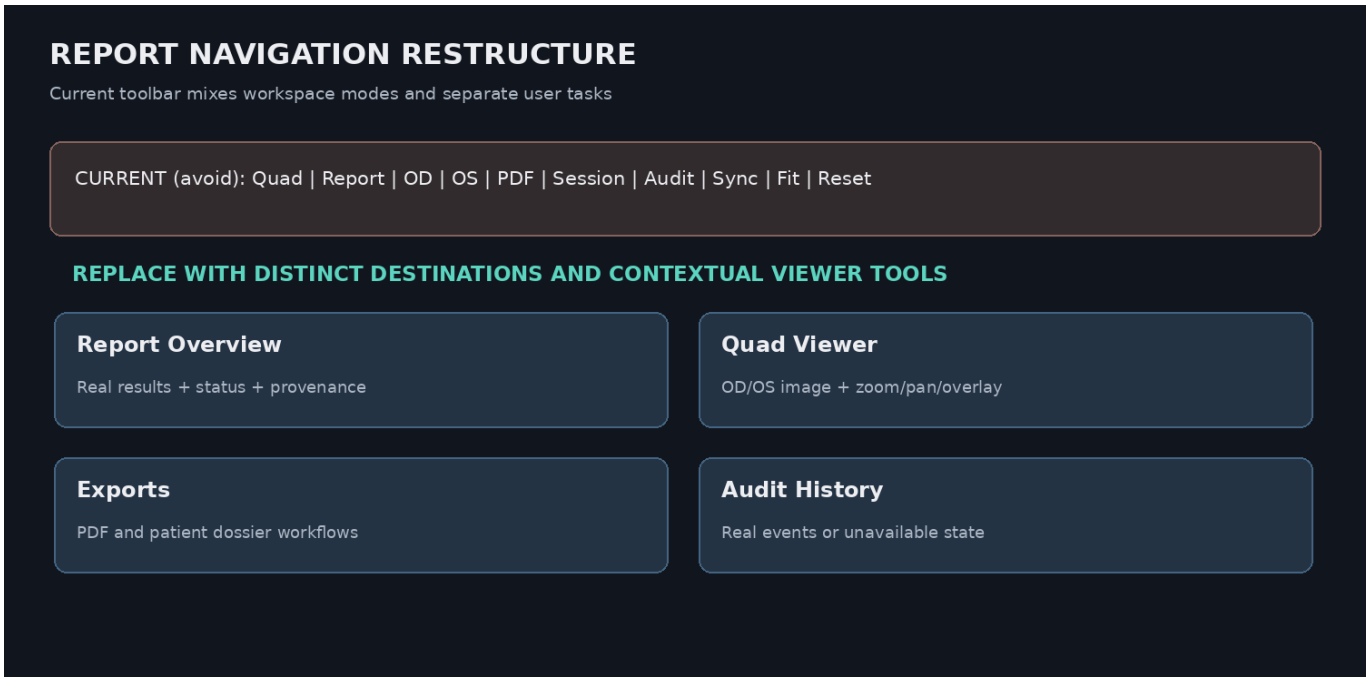
4. Design reference - Patient Profile and Examination Detail



Separate Patient Profile screen, accessible from the directory, with examination timeline and contextual actions.

Examination Detail must be another navigable screen with the selected patient and examination IDs, OD/OS image association, actual Stage I gate results when present, and Stage II/III unavailable states until genuine backend outputs exist.

5. Design reference - Report / PDF / Session / Audit



Keep image view controls near the Quad Viewer. Move PDF Export, Session Dossier and Audit History into routed workflow destinations.

This preserves functionality while reducing toolbar clutter. A small file-save confirmation modal is fine; a full PDF export workflow should not be hidden behind a single overstuffed toolbar button.

6. Separation of responsibilities

Antigravity - allowed	Claude - separate backend
Native GUI routes, layouts and states	HierarchiRetina inference / ONNX / CUDA
Patient directory and exam presentation	SQLCipher patient database and audit persistence
PDF Export / Audit pages and readiness UX	Actual PDF engine and durable audit APIs
Frontend tests / screenshots / Obsidian Frontend notes	Model validation, medical pipeline, backend tests

7. Current limitations to keep truthful

- Task 05 reported 94 GUI tests and built report/export/audit UI, but real PDF rendering and durable audit persistence were unavailable.
- The user requested improved navigation and page design, not permission to edit the backend or merge branches.
- Pinned egui/eframe versions differ across prior reports; inspect current Cargo.toml rather than assuming v0.31 or v0.37.
- The active backend was relocated to Hackthon Backend; do not blindly retain stale Dird relative path dependencies.

8. Implementation work packets

WP1 - Baseline inspection

Read canonical memory + feature docs; audit AppPage, current Patients list rendering, Report toolbar, state transitions and current git changes.

WP2 - Native route model

Add minimal typed patient/exam/report/export/audit route state with safe back stack and explicit ID ownership.

WP3 - Directory redesign

Build responsive list, actual search/filter/sort, preview panel and empty states; preserve existing fixtures only as marked test data.

WP4 - Profile and exam pages

Implement genuinely distinct screens with patient/exam context, timeline, OD/OS and report navigation.

WP5 - Report navigation cleanup

Preserve Quad + Report Overview; move major PDF, session and audit flows to proper destinations; keep controls contextual.

WP6 - Tests and desktop verification

Perform source tests, real Linux click-through screenshots, keyboard navigation and patient-isolation checks.

WP7 - Documentation and memory

Exact git-based file-change report, all evidence, linked Obsidian Frontend notes and Task 07 handoff.

9. Testing plan and 100% acceptance gate

Test category	Acceptance
Route behavior	Directory -> Profile -> Exam -> Report -> Back preserves selected IDs
Isolation	Switching patient never leaks another patient's images or results
Navigation	Viewer tools stay on viewer; PDF/Audit/Session have navigable destinations
Backend states	Real PDF/audit unavailable status, no fake Saved/Success
Responsive GUI	At 1280x820 and 700x650, no clipping or unusable empty region
Regression	Home/Chat/Quad viewer/Stage I readiness unaffected
Build quality	fmt + strict Clippy + tests + release build + git diff --check

Quality commands

```
cd "/home/claude/Desktop/Hackthon dezine/rust"
cargo fmt --all --check
cargo clippy --workspace --all-targets -- -D warnings
cargo test --workspace
cargo build --release
git diff --check
```

Task is COMPLETE only for genuine GUI screen architecture, interactions and verified screenshots. Backend PDF generation, database persistence, and unintegrated AI phases must remain independently BLOCKED / PENDING.

10. Full normal-text Antigravity implementation prompt

Copy the accompanying TXT file for convenient use. The complete prompt is reproduced below for a self-contained PDF.

RETINACARE / ANTIGRAVITY - TASK 06.1 UI ARCHITECTURE CORRECTION

DEDICATED PATIENT SCREENS + REPORT / EXPORT NAVIGATION CLEANUP

ROLE AND INTENT

You are the dedicated native Rust UI owner for RetinaCare. This task is a corrective GUI design and navigation pass following the Task 05 partial report and the Task 06 Patient Directory screenshot. The user does NOT want every workflow cramped inside one Report page or merely exposed as modal buttons. Deliver professionally designed, distinct application screens with persistent sidebar navigation and real back/breadcrumb interactions. You must not create multiple OS windows for each screen, web pages or backend code. Use egui/eframe and the existing AppPage/screen state architecture.

FIRST: ACKNOWLEDGE THE SCREENSHOTS ACCURATELY

Screenshot A: Report workspace top toolbar currently includes Quad, Report, OD, OS, PDF, Session, Audit, Sync, Fit and Reset. This mixes major workflows and local viewer controls.

Screenshot B: Patients already has a LEFT-SIDEBAR route and lists three clearly labeled SYNTHETIC test patients. It is not literally missing as a separate top-level screen. The actual problem is sparse, unfinished page composition, lack of genuine distinct patient-profile / examination-detail destinations, weak hierarchy and too much empty space.

Never claim a feature is absent merely because a screenshot did not show it; audit current Rust source first.

ABSOLUTE RULES

- Only native Rust egui/eframe. No HTML, CSS, JavaScript, TypeScript, Electron, Tauri, React or WebView.
- Antigravity owns GUI; Claude owns backend medical AI, database, model conversion and PDF engine.
- DO NOT edit backend source at the relocated location /home/claude/Desktop/Hackthon dezine/Hackthon Backend or older Dird checkout. Do not merge branches or repositories.
- Frontend root expected: /home/claude/Desktop/Hackthon dezine/rust. Verify instead of assuming.
- Read AGENTS.md, relevant Ponytail/Superpowers skills and the canonical Obsidian handoff index at /home/claude/Desktop/Hackthon dezine/memory BEFORE targeted code inspection.
- Preserve Tasks 01-05, working model integration, patient isolation, cancel/retry and current data models.
- No fake medical results, fabricated patient records, screenshots passed off as working features, fake PDF exports, or silent persistence.
- Never mark Task 05 fully complete while real backend PDF/audit APIs remain unavailable.

IMPLEMENTATION SCOPE: FIVE DISTINCT WORKFLOWS

A. PATIENT DIRECTORY SCREEN (retain existing left sidebar Patients navigation)

- Strong page heading, compact descriptive subtitle, searchable record list and meaningful whitespace.
- Use an organized master-detail layout at desktop widths: patient list + contextual preview or examination summary. At narrow widths, single-column layout with proper scrolling.

- Provide filters/sort when supported by actual records: patient code, display name, last exam, number of exams. Do not invent values, global metrics or fake patients in production.
- Clear, appropriately labeled Add Patient button only when supported; otherwise real form states / explicit unavailable feedback.
- Select a patient to navigate to a separate Patient Profile screen, not just expand a card in-place.

B. PATIENT PROFILE SCREEN

- Dedicated content with breadcrumbs (Patients / Patient), Back to Patients, patient metadata and examination timeline.
- Actions to view/create an examination through existing safe data contracts, no pretend saves.
- Preserve patient context when moving to Report and back. Show unavailable if persistence backend is not wired.

C. EXAMINATION DETAIL SCREEN

- Dedicated route for one exam: patient ID, examination ID, date if known, OD/OS, original retinal image thumbnail, Stage I actual inference results and Stage II/III honest unavailable badges.
- Open Report button navigates to the correct exam report, never to stale data.

D. REPORT WORKSPACE STRUCTURE

- Preserve both genuine Quad and Report Overview views, but do not treat PDF/Session/Audit as zoom-tool peers.
- Keep only contextual viewing controls near image viewport (OD, OS, Sync, Fit, Reset, panel mode).
- Make Report Overview a clearly named destination; full PDF Export, Session Dossier Export, and Audit History must have their own navigable sections or nested destinations, with Back/breadcrumb navigation.
- Modals are okay for brief confirmation, file picker, errors and destination selection; do not force primary workflows into a single crowded toolbar/modal.

E. EXPORT & AUDIT ROUTES

- PDF Export screen: real report preview, generation readiness, local save location and true backend unavailable messaging. No fake success if retinacare-reports is not implemented.
- Session Export screen: scope guard with patient + exam identity and secure local export actions. Distinguish JSON dossier export that truly works from PDF export that is not connected.
- Audit History screen: scoped actual persisted events when service exists; show truthful unavailable/empty states otherwise. Test mock is test-only and clearly labeled.
- Do not create unnecessary top-level sidebar items. Group secondary routes beneath Reports or Patients where appropriate; document the route map.

UI DESIGN DIRECTION

Premium charcoal background, calm neutral cards, modest teal/blue accent, clear hierarchy, aligned list columns, content max-width where useful, responsive 1280x820 and 700x650, meaningful density without filling every pixel. Avoid 3 rows of data at the top followed by a giant visually empty canvas; use a second contextual preview panel, examination timeline and helpful empty states instead of decorative cards. Show synthetic test labels prominently and never make fixture records look clinical.

STATE / ROUTING RULES

Use typed AppPage or Screen enums with structured navigation context. Separate patient_id, exam_id and image_id. On route transitions validate that the target record belongs to the active patient. Do not duplicate state and do not create a new database. Preserve sidebar, back navigation and examination-specific report context. Never allow cached results from patient A to appear on patient B's profile. No full redraw or loading side effects every egui frame. Resolve legacy Report modal state compatibility deliberately rather than deleting it blindly.

CONCRETE WORK PLAN

1. Inspect git status, AppPage enum, Patients rendering, Report toolbar, export/report state, existing tests and Task 05/06 documentation. Record observed vs reported behavior.

2. Define minimal route hierarchy and relevant data model adapters; write a small implementation plan with actual file paths.

3. Implement Patients Directory redesign and dedicated Patient Profile / Examination Detail screens.

4. Refactor Report navigation into distinct viewer/overview and export/audit destinations while retaining working controls.

5. Test real navigation and real backend-unavailable presentation. Mocks ONLY in unit tests/explicit development fixtures.

6. Launch native Ubuntu build and click through every screen. Capture real screenshots at desktop and narrow widths.

7. Run fmt, strict Clippy, tests and release build. Record warnings and skip/failure honestly.

8. Write exact file-by-file changes, task acceptance verdict and Obsidian memory updates.

REQUIRED TESTS

- Sidebar Patients nav opens directory and does not change identity unexpectedly.
- Search/filter/sort stable; empty/no-results state; large dataset scroll responsiveness.
- Directory -> Profile -> Exam Detail -> Report -> Back retains correct IDs.
- Patient A -> Patient B causes no cross-patient results or image bleed.
- OD/OS changes do not select a different exam silently.
- Report Quad / Report Overview mode remains functional.
- PDF Export and Audit screens show BackendUnavailable if not wired.
- Existing JSON dossier export works only when actual service succeeds; invalid scope rejected.
- Keyboard focus, Escape, back and modal dismissal; narrow screen controls not clipped.
- Task 01-05 regressions including live Stage I service readiness.

REQUIRED COMMANDS (from rust/)

cargo fmt --all --check

cargo clippy --workspace --all-targets -- -D warnings

cargo test --workspace

cargo build --release

git diff --check

FINAL ARTIFACTS

- docs/features/GUI-P06-patient-navigation-redesign/ audit, design, route-map, implementation plan, tests, screenshot evidence, file-change report and next-task handoff; reuse existing Task 06 docs when appropriate.
- Canonical Obsidian vault concise updates at /home/claude/Desktop/Hackthon dezine/memory (Frontend/ and Handoffs/ only, do not overwrite Claude's Backend/ notes); wiki-link updated architecture and tests.
- Screenshots: patients_directory_verified.png, patient_profile_verified.png, exam_detail_verified.png, report_navigation_verified.png, export_unavailable_verified.png, audit_unavailable_verified.png, narrow_window_verified.png. Save only states actually tested.

DEFINITION OF DONE

Dedicated patient/exam screens, professional directory design, clean Report/Export/Audit navigation, reliable back/breadcrumbs, privacy-safe context, real backend-unavailable states, 100% native Rust, no backend modification, no fake AI data, passing tests, real desktop screenshots, complete documentation. If persistence or PDF backend not available, mark those specific integration features BLOCKED and report an accurate PARTIAL task verdict; do not claim backend functionality finished.

START NOW. Implement the real screen architecture and visual redesign. Do not stop after an audit, do not rewrite completed medical AI code, do not start unrelated Task 07.

11. Complete JSON master prompt

The same machine-readable instruction is provided in a separate .json file. The full JSON is also reproduced here.


```
{
  "prompt_id": "RETINACARE-ANTIGRAVITY-GUI-TASK06-1-NAV-REDESIGN",
  "project": "RetinaCare",
  "agent": "Antigravity",
  "milestone": "Task 06.1: Dedicated Patients/Examination Screens and Report Workflow Navigation",
  "priority": "P0 - UI correction and navigation",
  "mode": "MEMORY_FIRST_AUDIT_PLAN_IMPLEMENT_TEST_VISUAL_VERIFY_DOCUMENT",
  "objective": "Replace sparse patient-directory presentation and modal-only/crowded report workflow
with distinct native egui screens and clean navigation, preserving the existing persistent sidebar and
Stage I backend integration.",
  "screenshot_ground_truth": {
    "patients_screenshot": "Patients is ALREADY a sidebar page. It shows three SYNTHETIC fixture patient
 rows near the top and excessive empty space. Improve this view instead of claiming no patients route
exists.",
    "report_screenshot": "Quad, Report, OD, OS, PDF, Session, Audit, Sync, Fit, Reset are crowded into
one toolbar. Separate major user workflows from view controls."
  },
  "read_first": [
    "AGENTS.md",
    "/home/claude/Desktop/Hackthon dezine/memory/01-claude-start-here.md",
    "/home/claude/Desktop/Hackthon dezine/memory/00-index.md",
    "Relevant Frontend and Handoffs notes",
    "GUI Task 05 and Task 06 implementation/handoff documents",
    "Applicable existing Ponytail and Superpowers skills"
  ],
  "paths": {
    "gui_root_expected": "/home/claude/Desktop/Hackthon dezine/rust",
    "backend_relocated": "/home/claude/Desktop/Hackthon dezine/Hackthon Backend",
    "obsidian_vault": "/home/claude/Desktop/Hackthon dezine/memory"
  },
  "ownership": {
    "allowed": "GUI Rust source, GUI tests, GUI docs and Frontend Obsidian notes",
    "prohibited": [
      "Backend source edits",
      "AI model changes",
      "Python production inference",
      "Model re-export",
      "Git branch merge",
      "Unauthorised commit/push",
      "Web frameworks or WebView"
    ]
  },
  "navigation_design": {
    "shell": "Keep one native Rust desktop window with the persistent left sidebar. Use distinct routed
content screens and breadcrumbs, not separate operating-system windows.",
    "main_routes": [
      "Home",
      "Patients",
```

```
      "Report",
      "Previous Chats",
      "Plugins",
      "Settings if already supported"
    ],
    "patient_subroutes": [
      "PatientDirectory",
      "PatientProfile(patient_id)",
      "ExaminationDetail(patient_id,examination_id)"
    ],
    "report_subroutes": [
      "ReportOverview(examination_id)",
      "QuadViewer(examination_id)",
      "PdfExport(examination_id)",
      "SessionDossierExport(patient_id,examination_id)",
      "AuditHistory(examination_id)"
    ],
    "ui_detail": "Keep OD/OS, Sync, Fit and Reset as contextual viewer controls rather than top-level
report-workflow tabs. Modals only for confirmations/dialogs."
  },
  "screen_requirements": {
    "patients_directory": [
      "Responsive master-detail composition with searchable list and contextual preview",
      "Actual record counts and dates only",
      "Sort/filter from supported fields",
      "Clearly labeled synthetic fixture records",
      "Real navigation to PatientProfile",
      "Useful empty/error states and no giant unusable blank area"
    ],
    "patient_profile": [
      "Dedicated page with breadcrumbs and Back",
      "Actual patient metadata",
      "Examination timeline and action to open exam",
      "No fake database save success"
    ],
    "examination_detail": [
      "Dedicated page for patient+exam",
      "OD/OS source-image association",
      "Actual Stage I results if available",
      "Truthful Stage II/III unavailable",
      "Open Report for the selected examination"
    ],
    "report_and_exports": [
      "Quad and overview remain distinct content modes/routes",
      "PDF export full screen or routed workspace, not only crowded toolbar modal",
      "Patient-scoped JSON export only when actually implemented",
      "Audit history route shows real data or BackendUnavailable",
      "No fake success confirmations"
    ]
  },
  "state_and_privacy": [
```

```
    "Typed navigation state with patient_id and examination_id",
    "Reject mismatched patient/exam pair",
    "Clear stale result during patient switching",
    "Preserve image and report context across Back navigation",
    "No unrelated app state reset",
    "No unencrypted new patient store"
  ],
  "implementation_plan": [
    "Read existing route/page code and git status",
    "Record baseline and route map",
    "Implement directory redesign",
    "Implement patient profile and exam detail routes",
    "Refactor report/export/audit navigation",
    "Add tests for each transition and unavailable state",
    "Launch real desktop and capture screenshots",
    "Run quality gates",
    "Update precise file-change report and Obsidian links"
  ],
  "quality_gates": {
    "working_directory": "rust/",
    "commands": [
      "cargo fmt --all --check",
      "cargo clippy --workspace --all-targets -- -D warnings",
      "cargo test --workspace",
      "cargo build --release",
      "git diff --check"
    ],
    "required_tests": [
      "route and back stack",
      "directory search/filter/sort",
      "patient/examination isolation",
      "report/quad mode retention",
      "OD/OS switching",
      "PDF unavailable truthful state",
      "session export scoped data",
      "audit unavailable truthful state",
      "keyboard accessibility",
      "small-window layout",
      "Task 01-05 regressions"
    ]
  },
  "visual_evidence": [
    "patients_directory_verified.png",
    "patient_profile_verified.png",
    "exam_detail_verified.png",
    "report_navigation_verified.png",
    "export_unavailable_verified.png",
    "audit_unavailable_verified.png",
    "narrow_window_verified.png"
  ],
  "documentation": {
```

```
    "folder": "docs/features/GUI-P06-patient-navigation-redesign/",
    "must_include": [
      "existing screenshot and code audit",
      "navigation route diagram",
      "implementation plan",
      "screen-by-screen requirements",
      "test results",
      "live screenshots",
      "exact file-change report",
      "known backend blockers",
      "next-task handoff"
    ],
    "memory": "Update canonical Obsidian Frontend/Handoffs notes and wiki-links without overwriting
Claude backend notes."
  },
  "acceptance": "Complete only when distinct screens and navigation work, previous GUI features remain
intact, privacy isolation passes tests, and real desktop screenshots validate the design. Mark backend-
dependent PDF/audit features as unavailable until Codex/Claude implements them. No fabricated clinical
data.",
  "final_instruction": "START the Task 06.1 correction NOW. Do not recreate the whole app or merely
repaint the existing single-page toolbar. Keep a persistent sidebar, introduce real dedicated routed
screens, redesign Patients, preserve genuine Stage I integration, run tests, visually verify and
document every changed file. GUI ONLY."
}
```

12. Required agent final report

- A. Screenshot and current source audit
- B. Source ownership / Git state
- C. Before-and-after page architecture
- D. Patients Directory design and functionality
- E. Patient Profile and Examination Detail behavior
- F. Report, PDF, Session and Audit routes
- G. Patient isolation and backend unavailability
- H. All test commands, counts and failures
- I. Genuine desktop screenshots, with paths
- J. Exact files changed and why
- K. Obsidian wiki-link updates
- L. Task 06.1 COMPLETE/PARTIAL/BLOCKED verdict and Task 07 handoff

Never claim full backend integration from GUI-only unit tests. Preserve the reported Task 05 partial limitations and document any unmet dependency explicitly.